GitHub workflow

GitHub Normal Workflow: Update Feature & Main Branch from Local

1⃣ Go to your local repo

cd "C:\Users\user\Documents\github weeklyprojects\DevOpsBasics Git GitHub Repo"

2⃣ Create or switch to your feature branch

```
# Create a new branch for your weekly/feature work
git checkout -b weekly-feature

# OR switch to an existing branch
git checkout weekly-feature
```

3⃣ Add or modify files locally

```
git add .
git commit -m "Add new files for week X"
```

This records your changes in your **feature branch**.

4⃣ Push your feature branch to GitHub

```
git push -u origin weekly-feature
```

- -u sets the upstream for future pushes.

5⃣ Merge your feature branch into main locally

```
# Switch to main branch
git checkout main

# Pull latest changes from remote main
```

```
git pull origin main

# Merge your feature branch into main
git merge weekly-feature
```

- If there are conflicts:
    1. Open the conflicted files and fix them
    2. Stage fixed files: git add <file>
    3. Complete the merge: git commit (if required)

6️⃣ Push main branch to GitHub

```
git push origin main
```

✅ **Tips**

- Feature branches = isolated work; main = stable branch.
- Use git pull to fetch remote updates before pushing to avoid errors.
- Only use git pull --rebase if you want a **clean linear history**; it's optional.
- You can create branches **anytime**, even before adding files.
- Repeat this workflow every week to keep feature and main branches updated.

This workflow works for **weekly updates**, feature additions, or any new files — no rebase required.

If you want, I can also make a **visual diagram** showing **main + feature branch flow**, which makes it easier to understand at a glance for future reference.

Do you want me to make that diagram?

```
 GitHub Remote
+----------------+
|               |
|      main      |
|    (stable)    |
|               |
+----------------+
      ^
      | git push
      |
Local main branch
+----------------+
|               |
|      main      |
|    (stable)    |
|               |
+----------------+
      ^
      | merge
      |
Feature branch (weekly-feature)
+----------------+
```

```
|              |
| weekly-feature |
|  (local work)  |
|              |
+----------------+
      ^
      | git add / commit
      |
Your working directory (local files)
```

Industry Standard GitHub Workflow

Feature Branch → Push → Pull Request → Merge

---

1 Clone the repository

```
git clone https://github.com/your-username/repo-name.git
```

---

2 Go into the repository

```
cd repo-name
```

---

3 Create a feature branch

Developers **never work directly on main**.

```
git checkout -b feature-branch-name
```

Example:

```
git checkout -b week-3-documentation
```

---

4 Add or modify files

```
git add .
```

---

5 Commit changes

```
git commit -m "Added Week 3 DevOps documentation"
```

---

6 Push the feature branch to GitHub

```
git push -u origin feature-branch-name
```

Example:

```
git push -u origin week-3-documentation
```

7️⃣ Create a Pull Request (on GitHub)

Go to your GitHub repository.

GitHub will show:

"Compare & Pull Request"

Click it and:

- Base branch → main
- Compare branch → feature-branch-name

Add a description and click:

Create Pull Request

8️⃣ Merge Pull Request

After review (or by yourself for personal projects):

Click:

Merge Pull Request

GitHub will merge the feature branch into main.

9️⃣ Update your local main branch

After merging on GitHub:

```
git checkout main
git pull origin main
```

Now your local repo is updated.

🔟 (Optional) Delete the feature branch

After merging, the feature branch is usually deleted.

Local:

```
git branch -d feature-branch-name
```

Remote:

```
git push origin --delete feature-branch-name
```

Complete Professional Workflow

```
git clone repo-url
cd repo-name

git checkout -b feature-branch
git add .
git commit -m "update"
git push -u origin feature-branch

# Create Pull Request on GitHub

git checkout main
git pull origin main
```

Why companies use this workflow

- ✅ Code review
- ✅ Team collaboration
- ✅ CI/CD pipelines trigger on PR
- ✅ Prevents breaking main branch
- ✅ Better commit history

DevOps Interview Tip

If asked:

"What Git workflow do you use?"

Answer like this:

"We follow a feature branch workflow. Developers create a feature branch from main, commit changes, push the branch, and create a Pull Request. After code review and CI checks, the PR is merged into main."

That answer sounds **very professional** in interviews.